

Evaluación de la calidad de InvenTree mediante pruebas de software multinivel: Estudio de caso en un Sistema de Gestión de Inventarios de Código Abierto

Tejada Lazo Jordy Rolando¹, Hurtado Bejarano Michael Steve², Yare Chulunquia Kevin Pedro³, Jaita Chura José Manuel⁴, Del Castillo Montoya Christopher Brad⁵

Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín

jtejadala@unsa.edu.pe, mhurtadob@unsa.edu.pe, kyare@unsa.edu.pe, jjaitac@unsa.edu.pe, cdelcastillo@unsa.edu.pe

Abstract— Actualmente, en el área del software testing, la complejidad de los sistemas exige estrategias capaces de combinar distintos niveles de validación y automatización continua. En el presente trabajo se diseñó y aplicó una estrategia integral de pruebas de software al sistema de gestión de inventarios de código abierto InvenTree v1.3.0. El objetivo fue evaluar el comportamiento de sus componentes y flujos críticos mediante pruebas unitarias, funcionales, de integración, de sistema y de aceptación. El proceso se organizó en tres hitos que comprendieron planificación, diseño de casos, preparación del entorno, ejecución, registro de incidencias, trazabilidad y automatización continua. Se analizaron principalmente los módulos de partes, stock, órdenes, fabricación, empresas, usuarios, permisos, reportes y etiquetas. Para diseñar los escenarios se empleó partición de equivalencia, análisis de valores límite, tablas de decisión y transición de estados, combinando pruebas automatizadas y manuales según las características de cada nivel. La ejecución obtuvo una cobertura global de código del 86,91 %, lo que permitió determinar el alcance de la suite e identificar componentes que requieren validaciones adicionales. Asimismo, la integración con GitHub Actions permitió automatizar la ejecución de las pruebas y condicionar las etapas posteriores del pipeline a sus resultados. Los hallazgos muestran que la cobertura debe interpretarse junto con la calidad de los casos y la evaluación de reglas de negocio, ya que no garantiza por sí sola la ausencia de defectos. Se concluye que la estrategia aplicada proporcionó un proceso organizado, reproducible y trazable para evaluar un sistema empresarial real.

Keywords— automatización de pruebas, cobertura de código, GitHub Actions, InvenTree, pruebas de software.

I. INTRODUCCIÓN

La creciente complejidad de los sistemas de software ha propiciado la necesidad de llevar a cabo procesos que verifiquen su comportamiento, los errores que contengan y la minimización de los riesgos asociados al funcionamiento de los mismos. Las aplicaciones actuales se apoyan en un conjunto de tecnologías como las interfaces Web, los servicios, las bases de datos, los componentes externos y las reglas de negocio que han de evaluarse de manera conjunta. Bertolino mantiene que las pruebas son "una actividad básica registrada en la ingeniería de software" [1], pero también advierte que a pesar de su uso generalizado, pueden verse

como costosas, imprevisibles, poco sistemáticas y con efectos difíciles de calcular [1]. Por este motivo, la calidad de un producto no solo dependerá de practicar una gran cantidad de casos sino que deberemos diseñar en su lugar una estrategia coherente y en función de su arquitectura, riesgos y necesidades de negocio.

La automatización tiene, además, una particular relevancia si tenemos en cuenta la necesidad de ejecutar pruebas que sean repetibles a la vez que nos aporten toda la retroalimentación que requerimos cada vez que aplicamos una modificación al software. Garousi y Mäntylä subrayan que "una estrategia de automatización puede fallar cuando no se intenta aplicar el momento, contexto y enfoque correctos" [2]. De esta forma, la automatización contendrá una selección razonada de módulos, técnicas y niveles de prueba y mantendrá aquellas actividades que sean manuales cuando requieran la interpretación humana. La inclusión de la integración continua es una progresión que se incorpora porque "los sistemas de integración continua son capaces de automatizar la construcción, la compilación y la prueba del software" [3]. Integrar estas prácticas entre sí permitirá ejecutar comprobaciones con frecuencia, detectar regresiones y conllevar la preservación de evidencia sobre los resultados obtenidos mediante su desarrollo.

Los proyectos de código abierto son espacios adecuados para investigar estas prácticas, ya que cuentan con el código fuente, los casos de prueba, el historial de cambios y los pipelines que son usados por las comunidades [2]. Según Shahin, Babar y Zhu las prácticas de integración, entrega y despliegue continuos afinan las funcionalidades «de forma frecuente y fiable» [4]. Además, Wang et al. también afirman que «los mayores niveles de evolución de la automatización se asocian con una mayor calidad del producto» [5]. Sin embargo, estos beneficios dependen de seleccionar pruebas adecuadas, de la reproducibilidad del entorno y del uso de comprobaciones para detener el avance hacia la entrega en módulos con resultados no satisfactorios.

Con el presente trabajo pretendemos diseñar, aplicar y documentar un enfoque holístico de pruebas sobre InvenTree v1.3.0, un sistema de gestión de inventarios de código abierto. El trabajo escrito tiene como propósito explorar el comportamiento de sus componentes y flujos críticos por medio de pruebas unitarias, funcionales, de integración, de sistema y de aceptación de los casos de prueba. Para ello

consideramos los módulos de gestión de partes, administración del stock, órdenes de compra, órdenes de venta, fabricación, empresas, usuarios, permisos, reportes y etiquetas. También utilizamos técnicas como partición de equivalencias (equivalence partitioning), análisis de límites (boundary value analysis), tablas de decisión y transición de estados, para evaluar los trazados válidos, los límites y las operaciones incompatibles con los estados del sistema.

Con una aproximación gradual el proceso fue organizado en tres hitos que abarcaban la planificación, el diseño, la ejecución, la gestión de defectos, la trazabilidad y la automatización por medio de GitHub Actions. La ejecución de las pruebas arrojó una cobertura de código de 86,91 %, lo que permitió determinar el alcance de la suite respecto de los componentes donde fue aplicada e identificar componentes que requieren casos adicionales. La aplicación del pipeline establecido por lo tanto permitió automatizar la ejecución de las pruebas y condicionar las etapas posteriores a su resultado. En consideración de ello, la contribución más destacada del trabajo consiste en ofrecer una experiencia trazable de la aplicación de pruebas multinivel en un sistema real que integre planificación, ejecución, medición de cobertura y automatización continua.

II. ANTECEDENTES

A. Estado del arte de las pruebas de software en proyectos de código abierto

En el presente capítulo se expone que la actividad de ejecutar pruebas de software es fundamental para descifrar el comportamiento de un sistema, contrastar su comportamiento frente a la especificación de requisitos y encontrar defectos antes de que puedan llegar a los usuarios. En esta línea de pensamiento, Bertolino menciona que "las pruebas son una actividad primordial en la ingeniería del software" [1]. Sin embargo, la autora advierte que, aunque son usadas en la industria, su uso puede continuar siendo costoso, poco sistemático y de eficacia no predecible [1]. El problema es más pronunciado en las aplicaciones empresariales, que constan de numerosos módulos, servicios, reglas de negocio y fuentes de datos, donde un defecto en un solo módulo puede propagarse y afectar el comportamiento global del sistema, y explican que las pruebas no deben ser vistas como una fase del desarrollo sino tan solo como un proceso planificado que forman parte del mismo.

Como respuesta a estas dificultades, la automatización se ha convertido en una de las principales líneas de evolución para las pruebas de software. Garousi & Mäntylä indican que "las pruebas automatizadas de software consisten en la automatización de actividades de prueba" [2], no obstante su revisión de 78 fuentes confirma que decidir qué automatizar y en qué momento depende de cuestiones relacionadas con el sistema bajo análisis, con el diseño de los casos, las herramientas y las condiciones organizativas [2]. También advierten que una estrategia de automatización puede ser fallida si no se utiliza en el momento y en el contexto adecuado y con la metodología correcta. De ahí que cuando se

automatiza el proceso de prueba no sea trasladar sin más todas las pruebas manuales a una herramienta, sino seleccionar las actividades que se repiten y que tienen el mayor valor, sobre todo las pruebas de regresión, mientras se deja el espacio para validaciones manuales donde pueden ser exploraciones, interpretaciones o evaluaciones de un usuario.

Los proyectos de código abierto ofrecen un ámbito propicio para investigar estas prácticas, dado que sus repositorios permiten la revisión del código fuente, de los casos de prueba, del historial de cambios del proyecto y de los resultados de las ejecuciones automáticas. Hilton et al. afirman que «los sistemas de integración continua automatizan la compilación, construcción y prueba del software» [3]. Es un estudio en el que analizaron 34 544 proyectos alojados en GitHub, 1 529 291 ejecuciones de construcción y las respuestas de 442 desarrolladores. Los resultados concluyeron que había una creciente adopción de la integración continua y que los proyectos que la utilizaban publicaban sus versiones con una frecuencia dos veces superior a aquellos que no la utilizaban [3]; por tanto, la disponibilidad pública de los repositorios no solo impulsa la colaboración, sino que también permite obtener información verificable de la relación entre las pruebas automatizadas, la integración del cambio y la evolución del propio software.

La introducción de pruebas en procesos de integración, entrega y despliegue continuos ha ampliado el rol de las pruebas en el ciclo de desarrollo en sí. Shahin, Babar y Zhu consideran estas prácticas como mecanismos que «ayudan a las organizaciones a publicar frecuentemente y de manera fiable nuevas funcionalidades y productos» [4]. En su revisión sistemática de 69 estudios, identificaron 30 enfoques y herramientas orientadas a la reducción del tiempo de construcción y de prueba, a la visibilidad de resultados, a la ejecución continua de pruebas y a la detección de fallos en el pipeline [4]. También identificaron factores críticos como son el esfuerzo y el tiempo de prueba, la infraestructura, la transparencia del equipo y la experiencia técnica. Por lo tanto, la existencia de un pipeline no asegura por sí mismo la calidad del producto, pues también requiere de casos de prueba apropiados, de entornos reproducibles y de criterios que hagan imposible la continuidad del despliegue a partir de las validaciones anteriores en caso de identificar errores.

Por el contrario, Wang et al. analizaron la madurez de la automatización en 37 proyectos de código abierto desarrollados en java y corrieron complementando las respuestas obtenidas con datos extraídos de sus repositorios [5]. Los autores concluyeron que "los niveles de automatización más altos están asociados con una calidad más alta del producto" [5], obteniendo un nivel de significación de 0,000624. Además, un alto nivel de madurez de automatización se correlacionó con ciclos de publicación más cortos, alcanzando un $p=0,01891$ [5]. Lo que la investigación no implica es que automatizar más casos produzca un software automáticamente de calidad, sino que aplicar regularmente buenas prácticas, elegir adecuadamente las pruebas y enlazarlas, a su vez, a los procesos de integración continua puede mejorar la estabilidad y frecuencia de liberación del producto.

En síntesis, las investigaciones analizadas muestran una evolución desde actividades de prueba aplicadas de forma aislada a estrategias planificadas, repetibles e integradas a los procesos de automatización. A su vez, la literatura también muestra que su efectividad depende del contexto y de la madurez del proceso, así como de tener un equilibrio entre pruebas manuales y automatizadas [2], [4], [5]. Desde esta perspectiva, vemos recomendable analizar la aplicación de una estrategia multinivel sobre un sistema empresarial real de código abierto. Así, en este trabajo analizamos InvenTree por medio de pruebas unitarias, funcionales, de integración, de sistema y de aceptación, complementadas por un flujo de integración continua y con la finalidad de obtener evidencias trazables respecto al comportamiento de sus componentes, reglas de negocio y procesos críticos.

B. Estrategia de pruebas multinivel y automatización continua

Las pruebas multinivel son un tipo de estrategia de pruebas de software que distribuyen la verificación del software a través de diferentes niveles de manera que cada uno de los niveles, examine riesgos y comportamientos concretos. Ammann y Offutt afirman que «las pruebas son el principal medio por el que la industria evalúa el software durante el desarrollo» [6]. En el nivel unitario se efectúan verificaciones de funciones, clases o componentes de forma aislada; las pruebas de integración, por su parte, se encargan de garantizar la comunicación entre módulos, interfaces, servicios y fuentes de datos. Las pruebas de sistema verifican todo el software mediante flujos de extremo a extremo. Finalmente, las pruebas de aceptación se definen como aquellas que garantizan que el comportamiento observable satisfice las necesidades y criterios del usuario. La distribución entre niveles permite la detección de los defectos en el nivel más próximo a su origen y evitar así, una dependencia exclusiva de las validaciones generales al final del desarrollo.

La organización de los niveles de distribución es posible mostrarla con una estructura del tipo forma de pirámide, donde los niveles de pruebas unitarias conforman una amplia base de verificaciones rápidas, mientras que las pruebas de integración, sistema y aceptación son pruebas que conforman interacciones de cierta complejidad. Crispin y Gregory expresan que muchos equipos han encontrado en la pirámide de automatización «un modelo útil para planificar la automatización de sus pruebas» [7]. Sin embargo, este modelo no implica que los niveles superiores sean menos importantes. Se establece que sus casos deben seleccionarse con cuidado, debido a las características de un mayor número de componentes, de datos y de dependencias. Por lo que una estrategia equilibrada será la que combine pruebas unitarias frecuentes, pruebas de integración dirigidas y pruebas de sistema y de aceptación orientadas a los principales flujos del negocio.

La continua automatización hace posible repetir esta estrategia multinivel cuando se introduce cualquier cambio en el repositorio. Humble y Farley definen el pipeline de despliegue como una «implementación automatizada del proceso de construcción, despliegue, prueba y liberación de una aplicación» [8]. En este pipeline, cualquier cambio hace

que se ejecute una secuencia de compilación, configuración del entorno, ejecución de pruebas y generación de resultados. Si una determinada validación falla, el pipeline debe parar las etapas posteriores y retornar la información al equipo, evitando que una versión errónea o defectuosa llegue a ser desplegada. Esta organización coincide con Shahin, Babar y Zhu, quienes consideran la reducción del tiempo de prueba, la visibilidad de los resultados y el descubrimiento temprano de errores como propósitos esenciales de las prácticas continuas [4].

A pesar de sus ventajas, la automatización no elimina la necesidad de aplicar pruebas manuales. Garousi y Mäntylä expresan que «la automatización no puede sustituir por completo a las pruebas manuales» [2] ya que algunos escenarios precisan de la intervención humana, la exploración de la interfaz y la valoración del comportamiento desde el punto de vista del usuario. Por ello, en este estudio planteamos una estrategia combinada para InvenTree: las pruebas unitarias y parte de las pruebas de integración proporcionan controles técnicos repetibles, mientras que las pruebas funcionales, del sistema y aceptación permiten valorar las reglas de negocio y en su conjunto, flujos completos. Como se refleja en el Anexo A, estos niveles se integran dentro de un ciclo continuo en el que el avance hacia el despliegue queda condicionado por el resultado obtenido en las etapas precedentes.

III. CASO DE ESTUDIO

El trabajo que se presenta a continuación tiene su origen en un estudio de caso aplicado a un producto de software real. Runeson y Höst apuntan que el estudio de caso es una metodología adecuada para la ingeniería del software, pues "estudia fenómenos contemporáneos en su contexto natural" [9]. Siguiendo este enfoque, seleccionamos como objeto de estudio el sistema InvenTree porque es un software activo de código abierto, porque posee arquitectura modular, porque introduce reglas de negocio respecto a la gestión del inventario y porque dispone de una suite de pruebas y procesos de integración continua. Todo esto ha permitido llevar a cabo la evaluación del software en condiciones próximas a las de un proyecto real conservando el código, las dependencias, las configuraciones y la ejecución.

La propia documentación de InvenTree describe el sistema como "un sistema de gestión de inventarios de código abierto que proporciona una Administración intuitiva de partes y controlar existencias", indicando que se trata de un sistema que registra y clasifica partes, gestiona los artículos en stock, gestiona ubicaciones, proveedores y clientes, gestiona órdenes de compra y venta, planifica procesos de fabricación y mantiene la trazabilidad de los movimientos del inventario. Incorpora listas de materiales o Bill of Materials (BOM), reportes, etiquetas, permisos de usuario o una API REST para conectar con aplicaciones externas, su distribución bajo la licencia MIT favorece las inspecciones del código fuente, la creación de forks y la ejecución de pruebas sin tener que enfrentarse a las restricciones de un producto cerrado [11].

Respecto a su evolución, InvenTree se originó a partir de una aplicación que se consideraba que generaba sus contenidos principalmente en el servidor, haciendo uso del motor de plantillas de Django. La descripción del proyecto

hace referencia a este hecho indicando que: «InvenTree comenzó como una aplicación completa del lado del servidor» [12]. A medida que se fue incrementando la cantidad de información a gestionar, se pasó a incorporar una API REST realizada usando Django REST Framework, convirtiendo la interfaz de usuario y la gestión de los datos de la aplicación de modo que el contenido que se incorporaba en la misma se podía cargar dinámicamente desde el cliente [12]. Y después de eso, la interfaz fue actualizada mediante React y TypeScript, en tanto que el backend seguía ejecutándose mediante los desarrollos que se incorporaron en Python y Django. Estas características produjeron la estructura adecuada para aplicar pruebas en diversos niveles, dado que la arquitectura consta de interfaz web, API, lógica de negocio, modelos de datos, servicios asíncronos y persistencia en base de datos.

La revisión que fue considerada como sistema bajo prueba fue la de InvenTree v1.3.0, que fue publicada de forma oficial el 11 de abril de 2026 [13]. Esta versión pone de manifiesto un backend de Python y Django que proporciona modelos de negocio, acceso a datos, una interfaz administrativa y servicios REST. Para la tecnología del frontend se utiliza React y TypeScript; la persistencia se puede configurar mediante SQLite, PostgreSQL o MySQL. El sistema también puede utilizar Redis para caché y procesamiento de tareas; y por último, el sistema se puede desplegar mediante contenedores Docker. El repositorio oficial resume esta estructura indicando que «el núcleo de la aplicación InvenTree es un backend de base de datos desarrollado con Python y Django» [11]. La presencia de este tipo de capas permitieron analizar tanto funciones aisladas como la interacción entre la API, ORM, base de datos e interfaz de usuario.

Con el fin de circunscribir el alcance de la presente investigación, hemos determinado la existencia de ocho componentes funcionales que aparecen resumidos en el Anexo B. Dichos componentes funcionales recogen, entre otros, la mayor parte de las pruebas unitarias que se concentran de forma principal en los bloques Part, Stock, Order y Build, que son los cuatro bloques que tienen las reglas de negocio que permiten la creación de partes, la actualización de stock, la transición de estados y el procesamiento de pedidos. Las pruebas funcionales, de sistema y aceptación, que amplían el horizonte hacia autenticación, proveedores, clientes, permisos, reportes y etiquetas, han permitido el estudio de componentes aislados (test unitarios) y la validación de la interacción entre ellos mediante flujos completos de compra, fabricación, venta y control de inventario.

La selección de estos componentes estuvo en sintonía con el riesgo de alteración de la integridad del inventario. En efecto, una validación errónea en un pedido de compra modifica el stock disponible, de igual manera que un comando erróneo en una orden de producción afecta el consumo o asignación de las materias primas. Por lo tanto, la investigación no se limitó a testear funciones aisladas, sino que trabajó con las pruebas unitarias, funcionales, de integración, de sistema y de aceptación. Se empleó el fork InvenTree-VIUDANEGRA para mantener la configuración de trabajo y ejecutar, a partir de la suite elegida, un workflow de GitHub Actions. De esta forma se examinó InvenTree como lo

que es, un sistema integrado y no sólo como unos componentes aislados entre sí.

IV. METODOLOGÍA

El proceso de aseguramiento de calidad fue gestionado bajo una metodología ágil, empleando un tablero Kanban/Scrum mediante herramientas de control de versiones para la asignación y seguimiento de tareas. El ciclo de vida del proyecto se dividió en tres hitos (milestones) progresivos estructurados mediante Sprints:

- Hito 1 (Sprint 1): Centrado en la planificación inicial, la selección de InvenTree como sistema bajo prueba y la elaboración del plan de pruebas unitarias.
- Hito 2 (Sprint 2): Enfocado en la ejecución de las pruebas unitarias, el diseño de pruebas funcionales de caja negra y la planificación técnica de las pruebas de integración.
- Hito 3 (Sprints 3-4): Consolidación del proceso mediante la planificación de pruebas de sistema y aceptación, además de la integración del despliegue continuo (CI/CD) automatizado.

Para garantizar el flujo de información, la comunicación interna del equipo combinó métodos síncronos y asíncronos. Se utilizaron servicios de mensajería instantánea para la resolución de bloqueos urgentes y videollamadas para las reuniones diarias (Daily Stand-ups) y ceremonias de inicio/cierre de Sprint. La documentación técnica, el registro estructurado de tareas y el reporte de defectos se canalizaron formalmente a través de las herramientas integradas de gestión de incidencias y repositorios.

A. Entorno de Trabajo y Gestión de Artefactos

Para mantener un entorno de pruebas aislado y con trazabilidad completa, el equipo implementó una arquitectura basada en múltiples repositorios en la plataforma GitHub, centralizando los artefactos de la siguiente manera:

- Repositorio Original: Código oficial del proyecto InvenTree empleado como sistema base de código abierto.
- Repositorio de Pruebas (Fork): Bifurcación del repositorio principal, utilizada exclusivamente para la revisión técnica, análisis de código y ejecución local de las pruebas sin alterar el producto original.
- Repositorio de Grupo: Espacio centralizado para la gestión, organización y resguardo de todos los entregables académicos del equipo.
- Tablero de Proyecto (Project Board): Herramienta utilizada para la gestión ágil, permitiendo el seguimiento detallado de las tareas de cada hito.
- Página del Proyecto (Pages): Medio de presentación pública diseñado para exponer los objetivos, alcance y resultados finales de la investigación.
- Wiki de Documentación: Plataforma elegida para alojar la documentación técnica integral y navegable del plan de pruebas.

B. Tipos de Pruebas Utilizadas

La estrategia de evaluación se estructuró gradualmente para examinar el software desde sus componentes mínimos hasta los flujos completos de usuario. A nivel de organización y planificación, el proceso se definió de la siguiente manera:

- **Pruebas Unitarias:** Se planificó la re-ejecución y el análisis de la suite existente en InvenTree para validar de forma atómica y aislada los componentes del backend (Python/Django) y del frontend (React/TypeScript). Se estableció el uso de herramientas como el framework de pruebas de Django, el motor pytest y librerías de cobertura para garantizar el control sobre las métricas de ejecución en módulos críticos.
- **Pruebas Funcionales:** A nivel organizativo, se adoptó un enfoque de caja negra para evaluar el comportamiento externo del sistema frente a los requisitos observables, sin intervenir en el código fuente. El diseño de los escenarios se estructuró combinando técnicas formales de validación: Partición de Equivalencia (PE), Análisis de Valores Límite (AVL), Tablas de Decisión y Transición de Estados.
- **Pruebas de Integración:** La planificación se orientó a verificar la correcta comunicación entre las distintas capas arquitectónicas del sistema. Se mapearon las interacciones clave, tales como la comunicación de la API REST con los modelos del sistema, la persistencia con la base de datos y los flujos del pipeline de integración continua oficial del proyecto.
- **Pruebas de Sistema:** Se planificó la validación del software como un todo unificado, evaluándolo de extremo a extremo (E2E). La organización contempló el diseño de múltiples suites de pruebas orientadas a flujos completos de negocio, destacando un escenario denominado "Golden Path" para verificar el ciclo íntegro del sistema: crear catálogo, comprar, fabricar y vender.
- **Pruebas de Aceptación:** Se definió un enfoque de evaluación manual estructurado para validar que el sistema satisface las necesidades del negocio desde la perspectiva final del usuario. La planificación simuló roles reales (como administradores de inventario y operadores de almacén) para comprobar la usabilidad, la claridad de los mensajes del sistema y el cumplimiento de los criterios de aceptación en la interfaz web.

V. RESULTADOS

A continuación, presentamos los resultados del proyecto.

A. Pruebas Unitarias

La ejecución de las pruebas unitarias se enfocó en validar el comportamiento atómico de los componentes internos del sistema de gestión de inventarios, evaluando los módulos centrales del backend desarrollados en Python/Django (stock, part, order y company). Este proceso se llevó a cabo en un entorno local aislado basado en Linux, utilizando las herramientas automatizadas pytest y la librería coverage.py sobre una base de datos SQLite preconfigurada.

1) Resultados Generales de Ejecución

Durante el proceso de validación, se ejecutó un total aproximado de 526 casos de prueba en un lapso de 1 hora, 15 minutos y 59 segundos (ver Anexo C). El análisis del comportamiento del sistema arrojó los siguientes indicadores operativos:

- **Casos Aprobados:** Se superaron exitosamente 487 pruebas, lo que representa una tasa de éxito consolidada del 92.6% en los módulos analizados.
- **Casos Fallidos y Errores:** Se registraron 24 pruebas con estado de fallo (FAILED), 15 errores críticos de ejecución (ERROR) y 143 advertencias menores de validación.

2) Análisis de Cobertura de Código

El análisis de impacto procesó un total de 42,077 sentencias de código fuente en los componentes seleccionados. De este universo, se alcanzó a verificar exitosamente la ejecución de 31,018 sentencias, dejando 11,059 líneas de código sin evaluación directa por los escenarios de prueba provistos. Esto derivó en una cobertura global del 74%. Aunque esta métrica se ubica por debajo del objetivo mínimo esperado del 85% definido durante la planificación metodológica, constituye un indicador realista y evidenciable del grado de validación dentro del ambiente de pruebas local.

3) Evaluación de Fallos e Incidencias Técnicas

Un análisis exhaustivo sobre las 24 pruebas fallidas y los 15 errores demostró que estos no implican de manera directa defectos críticos en el código en producción del software.

- **Concentración de errores:** La totalidad de los errores críticos (15) se manifestaron de manera exclusiva en la validación del módulo de procesamiento de órdenes (order). Esto responde a una fuerte dependencia interna con los modelos lógicos del catálogo de partes (part), así como a la ausencia temporal de datos predefinidos de prueba (fixtures) en la base local.
- **Limitaciones del entorno:** Diversos fallos técnicos estuvieron estrictamente asociados a las limitaciones arquitectónicas del entorno de desarrollo utilizado. Las omisiones en configuraciones específicas, la falta de inicialización de los trabajadores en

segundo plano para tareas asíncronas y las discrepancias con el pipeline oficial de integración continua (CI/CD) derivaron en falsos negativos. Un ejemplo claro de esto fueron las pruebas de notificación de inventario (PartNotificationTest), las cuales fallaron debido a la ausencia local del servicio despachador de eventos asíncronos.

B. Pruebas de Integración

La fase de pruebas de integración tuvo como objetivo principal verificar la correcta comunicación e interoperabilidad entre las distintas capas arquitectónicas del sistema InvenTree, abarcando la interacción entre la API REST, los modelos de base de datos, el ORM y el sistema de extensiones (plugins).

1) Cobertura Global del Sistema

Durante la ejecución automatizada de esta suite, la herramienta de análisis procesó un volumen superior a las 54,000 sentencias de código, logrando validar exitosamente más de 48,000 de ellas.

Métrica principal: Se alcanzó una cobertura global del 88.92%(ver Anexo D), un indicador altamente positivo que demuestra la robustez de las interacciones internas del software.

2) Desempeño por Componentes

Al desglosar los resultados por módulos del sistema, se identificaron los siguientes comportamientos operativos (ver Anexo E):

- Módulos de alta fiabilidad: Los componentes transaccionales y de seguridad más críticos superaron ampliamente el umbral del 90% de cobertura. Los subsistemas encargados de la gestión de usuarios (users), el procesamiento de órdenes de compra/venta (order) y el control físico del inventario (stock) demostraron una alta estabilidad en las operaciones de cruce de datos y persistencia.
- Oportunidades de mejora: El módulo destinado a la fabricación y órdenes de ensamblaje (build) registró una métrica del 79.22%, señalando una brecha en la validación automatizada de sus flujos integrados.

3) Análisis de Brechas y Falsos Negativos

Para comprender la naturaleza de las sentencias no evaluadas, se aplicó un filtro de oportunidad enfocado en identificar los veinte archivos con menor índice de cobertura (ver Anexo F). El análisis de este reporte reveló que las brechas no residen en la lógica de negocio activa, sino en elementos estructurales pasivos:

- Archivos de inicialización: Ficheros de configuración de arranque del servidor (como wsgi.py) y herramientas de rendimiento (profiling.py).
- Servicios externos: Integraciones específicas de red que no aplican a una ejecución estándar, como los protocolos de autenticación LDAP.
- Historial de base de datos: Ficheros históricos generados automáticamente para las migraciones estructurales del sistema.

C. Pruebas Funcionales

La ejecución de las pruebas funcionales se realizó mediante un enfoque de caja negra, evaluando el comportamiento observable de InvenTree a través de su interfaz web y de la API REST. Los escenarios fueron diseñados mediante partición de equivalencia, análisis de valores límite, tablas de decisión y transición de estados. De los 75 casos funcionales diseñados para el Hito 2, se ejecutaron 52 casos correspondientes a autenticación, gestión de partes, control de stock, órdenes de compra y órdenes de fabricación. Se obtuvieron 50 resultados exitosos y 2 no exitosos, lo que representa una tasa global de aprobación del 96,15 % (Ver Anexo H).

En autenticación y gestión de partes se ejecutaron 15 casos, todos aprobados. Estos verificaron el inicio y cierre de sesión, el rechazo de credenciales inválidas, las restricciones de acceso sin autenticación y las operaciones de creación, consulta, modificación, búsqueda y eliminación de partes. En control de stock se ejecutaron 17 casos, con 16 resultados exitosos y uno no exitoso, alcanzando una aprobación del 94,12 %. En órdenes de compra y fabricación se ejecutaron 20 casos, de los cuales 19 fueron aprobados y uno presentó un resultado no satisfactorio, equivalente a una tasa de éxito del 95 %. Los diez casos correspondientes a fabricación finalizaron correctamente.

La ejecución permitió registrar ocho defectos y observaciones. Los dos defectos que produjeron casos no exitosos se relacionaron con la validación de cantidades límite. En una transferencia de stock, la API aceptó una cantidad superior a las existencias disponibles sin retornar un error de validación. De forma similar, el endpoint de recepción de órdenes de compra permitió registrar 9999 unidades para una línea solicitada de 10 unidades. Ambos comportamientos pueden generar inconsistencias en el inventario y evidencian la necesidad de validar que la cantidad transferida o recibida no supere la cantidad disponible o pendiente.

También se identificaron observaciones de severidad baja o informativa: creación de artículos de stock con cantidad cero, transferencias entre una misma ubicación, visualización temporal del texto «Part: undefined» durante la carga de una parte, mensajes redundantes en el inicio de sesión y respuestas poco descriptivas al intentar eliminar una parte activa. Estas observaciones no impidieron completar los flujos principales, pero representan oportunidades de mejora en las reglas de validación y en la experiencia de usuario.

En conjunto, los resultados muestran un comportamiento funcional estable en los módulos evaluados y confirman la

correcta aplicación de restricciones de autenticación y de los flujos CRUD principales. No obstante, los fallos encontrados mediante valores límite demuestran que la aprobación de los recorridos convencionales no garantiza el cumplimiento de todas las reglas de negocio. Por ello, se recomienda incorporar validaciones explícitas en los endpoints de transferencia de stock y recepción de compras, así como ampliar la ejecución hacia los casos pendientes de órdenes de venta, clientes, categorías, proveedores, listas de materiales y búsqueda global.

D. Pruebas de Sistema

Las pruebas de sistema permitieron evaluar InvenTree como una aplicación integrada, verificando la interacción entre la interfaz web, la API REST, la lógica de negocio y la base de datos. La ejecución comprendió 428 casos de prueba, de los cuales 418 fueron aprobados y 10 presentaron resultados fallidos, obteniéndose una tasa de aprobación de 97,66 % (Ver Anexo I).

Los casos fueron distribuidos entre los principales componentes del sistema. En los módulos funcionales se ejecutaron 399 pruebas: 134 correspondieron a gestión de partes, 132 a órdenes, 78 a fabricación, 36 a usuarios y autenticación, y 19 a empresas y proveedores. De estas pruebas, 389 fueron satisfactorias y 10 fallaron, equivalente a una aprobación funcional del 97,49 %.

El módulo de órdenes obtuvo el mejor resultado entre los componentes de mayor tamaño, con 132 de 132 pruebas aprobadas. El módulo de empresas y proveedores también alcanzó una aprobación completa, con 19 de 19 pruebas exitosas. En gestión de partes se aprobaron 130 de 134 pruebas, mientras que en fabricación se aprobaron 73 de 78. En usuarios y autenticación se superaron 35 de 36 casos.

Además, se ejecutaron 29 pruebas de sistema no funcionales y transversales. Las 18 pruebas de seguridad, las 6 pruebas de permisos y trazabilidad y las 5 pruebas de desempeño fueron aprobadas en su totalidad, alcanzando una tasa de éxito del 100 % en estas tres categorías. Las pruebas de desempeño comprobaron que las operaciones evaluadas respondieron dentro de los umbrales establecidos.

Las pruebas automatizadas de extremo a extremo fueron incorporadas durante la ejecución por indicación posterior del docente. Por esta razón, el número de casos reportado es superior al contemplado inicialmente en el plan y representa el alcance real alcanzado por el equipo.

E. Pruebas de Aceptación

Durante esta fase de pruebas, se diseñó y ejecutó de manera continua un total de 55 casos de prueba distribuidos a lo largo de los 8 subprocesos de negocio (SP-1 a SP-8) estipulados en la planificación del plan de prueba donde el sistema logró satisfacer con éxito 50 de los escenarios evaluados, alcanzando una tasa de éxito global del 91% (Ver Anexo G). Este indicador supera el umbral mínimo del 90% definido como condición de éxito para el proceso logístico,

demostrando la alta madurez funcional de la plataforma en la mayoría de sus flujos comerciales end-to-end.

A pesar de una alta aprobación general, se identificó vulnerabilidades lógicas concentradas en Control de Stock (SP-2), donde los casos 6, 7 y 8 fallaron al no bloquear transferencias de montos superiores al stock real, permitir lotes fantasma con saldo cero y procesar traslados redundantes en React. En contraste, la falla en Gestión de Catálogo (SP-1) al permitir SKU duplicados no rompe el sistema directamente pero genera confusiones operativas humanas, mientras que la brecha en Órdenes de Compra (SP-3) al emitir órdenes vacías huérfanas resulta ser la de menor impacto directo para el negocio.

VI. CONCLUSIONES

La aplicación de una estrategia de pruebas multinivel ha permitido llevar a cabo la evaluación de InvenTree desde una perspectiva distinta, no entendida únicamente como un conjunto de funciones sin relación entre sí. En efecto, las pruebas unitarias han permitido comprobar la existencia de componentes contundentes, pero las pruebas funcionales, de integración, de sistema y de aceptación han permitido evaluar reglas de negocio, comunicación existente entre componentes y flujos de usuario completos. Esta combinación de técnicas nos ha dado una imagen más global del comportamiento del sistema, al tiempo que demuestra que cada nivel de prueba permite el descubrimiento de distintos tipos de problemas.

La ejecución de la suite alcanzó un porcentaje de código cubierto del 86,91 %, un resultado que permite comprobar que un número considerable de componentes seleccionados fue ejercitado durante la fase de pruebas. Sin embargo, este porcentaje no se puede interpretar, en ningún caso, como ausencia de fallos ni como garantía absoluta de calidad, entendiendo que la cobertura de un porcentaje sólo indica qué partes del código se ejecutaron, pero no conlleva, por sí sólo, la suficiencia de las validaciones desarrolladas; por ello, en su interpretación deben acompañarse con la calidad de los casos, la existencia de resultados esperados y la evaluación de situaciones límite.

Las técnicas de partición de equivalencia, análisis de valores límite, tablas de decisión y transición de estados han permitido elaborar casos que hacían referencia a condiciones válidas, inválidas y excepcionales. La aplicación de las mencionadas técnicas ha resultado de especial interés en aquellos módulos donde la operación depende de cantidades, permisos y estados previos, como son los casos de las partes, stock, órdenes de compra, ventas y fabricación; por ello, han permitido comprobar que aquellos escenarios alternativos y las restricciones de negocio requieren la misma atención que los recorridos principales del sistema.

La inclusión de GitHub Actions posibilitó poner en marcha la ejecución de pruebas mediante un flujo automatizado y reproducido. El pipeline que se diseñó condicionó las posteriores etapas a los resultados de las pruebas, de tal manera que es imposible continuar con el despliegue cuando las validaciones previas no terminan con éxito. Para resumir, el trabajo ejecutado sobre InvenTree sirvió para llegar a hacer

realidad un proceso de pruebas ordenado, documentado y trazable, alternando planificación, diseño, ejecución, medición de la cobertura y automatización continua sobre un proyecto concreto de código fuente abierto.

VII. TRABAJOS FUTUROS

Como trabajos futuros podemos considerar los siguientes:

- En el campo de pruebas de aceptación, desarrollar nuevos casos de prueba orientados a corregir los fallos críticos detectados en la interfaz de React; específicamente, el bloqueo estricto de transferencias que superen el stock real, la erradicación de lotes fantasma con saldo cero y la eliminación de traslados redundantes.
- Para corregir los falsos positivos, implementar y configurar trabajadores en segundo plano (workers) y un despachador de eventos asíncronos en el entorno local de pruebas, con el propósito de viabilizar la ejecución exitosa de componentes específicos como las pruebas de notificación de inventario (PartNotificationTest)
- Sobre robustez y la provisión de datos de prueba, desarrollar un mecanismo centralizado de inicialización de datos para mitigar de manera definitiva la fuerte dependencia interna entre los módulos de procesamiento de órdenes (order) y el catálogo de partes (part)
- Sobre las pruebas de integración, desarrollar una suite complementaria orientada específicamente a la lógica de negocio del módulo de fabricación y órdenes de ensamblaje (build), el cual registró un rendimiento del 79.22%, situándose por debajo del promedio de alta fiabilidad (>90%) alcanzado por los componentes de usuarios, órdenes y stock.

REFERENCIAS

[1] A. Bertolino, "Software testing research: Achievements, challenges, dreams," in Proc. Future of Software Engineering (FOSE '07), Minneapolis, MN, USA, 2007, pp. 85–103, doi: 10.1109/FOSE.2007.25.

[2] V. Garousi and M. V. Mäntylä, "When and what to automate in software testing? A multi-vocal literature review," Information and Software Technology, vol. 76, pp. 92–117, Aug. 2016, doi: 10.1016/j.infsof.2016.04.015.

[3] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE), Singapore, 2016, pp. 426–437, doi: 10.1145/2970276.2970358.

[4] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," IEEE Access, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2685629.

[5] Y. Wang, M. V. Mäntylä, Z. Lju, and J. Markkula, "Test automation maturity improves product quality—Quantitative study of open source projects using continuous integration," Journal of Systems and Software, vol. 188, Art. no. 111259, Jun. 2022, doi: 10.1016/j.jss.2022.111259.

[6] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2016, doi: 10.1017/9781316771273.

[7] L. Crispin and J. Gregory, More Agile Testing: Learning Journeys for the Whole Team. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2014.

[8] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley Professional, 2010.

[9] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," Empirical Software

Engineering, vol. 14, no. 2, pp. 131–164, Apr. 2009, doi: 10.1007/s10664-008-9102-8.

[10] InvenTree Project, "InvenTree documentation," InvenTree.

[11] InvenTree Project, "InvenTree: Open source inventory management system," GitHub. Accessed: Jul. 19, 2026. [Online].

[12] InvenTree Project, "User interface refactor," InvenTree Blog, Aug. 28, 2023.

[13] InvenTree Project, "Release notes: InvenTree 1.3.x," InvenTree Documentation, 2026.

VIII. ANEXOS



Anexo A – Modelo conceptual de una estrategia de pruebas multinivel integrada con automatización continua.

ID	Componente	Funcionalidades consideradas
SP-1	Gestión de partes	Creación, edición, consulta y eliminación de partes, categorías, parámetros, imágenes y BOM.
SP-2	Control de stock	Artículos almacenados, ubicaciones, ajustes, transferencias, conteos y trazabilidad.

SP-3	Órdenes de compra	Proveedores, líneas de compra, recepción de mercancía y cambios de estado.
SP-4	Órdenes de venta	Clientes, asignación de existencias, despacho y finalización de órdenes.
SP-5	Órdenes de fabricación	Creación de órdenes, asignación de materiales, consumo de stock y finalización.
SP-6	Empresas y proveedores	Gestión de proveedores, clientes, fabricantes, contactos y precios.
SP-7	Usuarios y permisos	Autenticación, roles, privilegios y restricciones de acceso.
SP-8	Reportes y etiquetas	Generación de documentos, etiquetas y salidas asociadas al inventario.

Anexo B. Componentes analizados del proyecto de código abierto Inventree.

InvenTree/stock/migrations/0117_stockitemtracking_part_alter_stockitemtracking_item.py	3	0	100%
InvenTree/stock/migrations/0118_auto_20200905_1210.py	34	12	65%
InvenTree/stock/migrations/0119_alter_stockitemtestresult_date.py	5	0	100%
InvenTree/stock/migrations/_init_.py	0	0	100%
InvenTree/stock/models.py	1183	247	79%
InvenTree/stock/serializers.py	586	58	90%
InvenTree/stock/status_codes.py	42	0	100%
InvenTree/stock/tasks.py	49	28	43%
InvenTree/stock/test_api.py	1084	20	98%
InvenTree/stock/test_migrations.py	161	0	100%
InvenTree/stock/test_views.py	59	0	100%
InvenTree/stock/tests.py	889	23	97%
InvenTree/users/_init_.py	0	0	100%
InvenTree/users/admin.py	49	7	86%
InvenTree/users/api.py	283	88	61%
InvenTree/users/apps.py	25	2	92%
InvenTree/users/authentication.py	19	9	53%
InvenTree/users/migrations/0001_initial.py	6	0	100%
InvenTree/users/migrations/0002_auto_20200804_0150.py	4	0	100%
InvenTree/users/migrations/0003_auto_20200805_2227.py	4	0	100%
InvenTree/users/migrations/0004_auto_20210113_1909.py	4	0	100%
InvenTree/users/migrations/0005_owner_model.py	5	0	100%
InvenTree/users/migrations/0006_alter_ruleset_name.py	4	0	100%
InvenTree/users/migrations/0007_alter_ruleset_name.py	5	0	100%
InvenTree/users/migrations/0008_apitoken.py	7	0	100%
InvenTree/users/migrations/0009_auto_20231020_2350.py	4	0	100%
InvenTree/users/migrations/0010_alter_apitoken_key.py	6	0	100%
InvenTree/users/migrations/0011_auto_20240523_1646.py	7	0	100%
InvenTree/users/migrations/0012_alter_ruleset_can_view.py	4	0	100%
InvenTree/users/migrations/0013_migrate_nfa_20240408_1659.py	21	7	67%
InvenTree/users/migrations/0014_userprofile.py	11	1	91%
InvenTree/users/migrations/0015_alter_useprofile_type.py	4	0	100%
InvenTree/users/migrations/_init_.py	0	0	100%
InvenTree/users/models.py	241	49	80%
InvenTree/users/oauth2_scopes.py	11	0	100%
InvenTree/users/permissions.py	68	5	93%
InvenTree/users/ruleset.py	24	1	96%
InvenTree/users/serializers.py	173	66	62%
InvenTree/users/tasks.py	85	16	81%
InvenTree/web/_init_.py	0	0	100%
InvenTree/web/templatetags/_init_.py	0	0	100%
InvenTree/web/templatetags/spa_helper.py	36	22	39%
InvenTree/web/utils.py	0	0	100%
TOTAL	42077	11659	74%
(.venv) ftyy@pop-os:~/Documentos/Inventree/src/backenes			

Anexo C. Cobertura global de Pruebas de Pruebas Unitarias

=====
ANÁLISIS DETALLADO DE COBERTURA DE PRUEBAS DE INTEGRACIÓN
=====
TOTAL DEL PROYECTO

Sentencias totales : 54,348
Sentencias faltantes: 6,021
COBERTURA GLOBAL : 88.92%

Anexo D. Cobertura global de Pruebas de Integración

MÓDULO	SENTENCIAS	FALTANTES	COBERTURA
InvenTree	8,246	1,033	87.47%
plugin	7,699	840	89.09%
part	7,443	864	88.39%
order	6,748	591	91.24%
stock	5,696	508	91.08%
common	5,285	473	91.05%
build	3,961	823	79.22%
report	2,602	259	90.05%
company	1,794	185	89.69%
users	1,493	51	96.58%
importer	1,157	177	84.76%
machine	1,142	103	90.98%
generic	696	43	93.82%
data_exporter	279	43	84.59%
web	94	26	72.34%
root	13	2	84.62%

Anexo E. Desempeño de módulos

ARCHIVO	STMTS	MISS	COV %
src/backend/InvenTree/InvenTree/profiling.py	6	6	0.00%
src/backend/InvenTree/InvenTree/wsgi.py	2	2	0.00%
src/backend/InvenTree/part/events.py	2	2	0.00%
src/backend/InvenTree/InvenTree/setting/ldap.py	26	24	7.69%
src/backend/InvenTree/build/tests.py	156	134	14.10%
...nd/InvenTree/part/migrations/0109_auto_20230517_1048.py	81	62	23.46%
...ckend/InvenTree/plugin/builtin/exporter/bom_exporter.py	144	108	25.00%
...nd/InvenTree/part/migrations/0111_auto_20230521_1350.py	55	41	25.45%
...InvenTree/company/migrations/0019_auto_20200413_0642.py	73	54	26.03%
...nd/InvenTree/part/migrations/0138_auto_20250718_2342.py	41	30	26.83%
...enTree/company/migrations/0036_supplierpart_update_2.py	47	34	27.66%
...d/InvenTree/stock/migrations/0106_auto_20240207_0353.py	57	40	29.82%
...d/InvenTree/stock/migrations/0108_auto_20240219_0252.py	40	28	30.00%
...nd/InvenTree/part/migrations/0102_auto_20230314_0112.py	46	32	30.43%
src/backend/InvenTree/plugin/builtin/labels/label_sheet.py	108	75	30.56%
src/backend/InvenTree/common/setting/tests.py	22	15	31.82%
...d/InvenTree/stock/migrations/0118_auto_20260205_1218.py	34	22	35.29%
...d/InvenTree/order/migrations/0052_auto_20211014_0631.py	28	18	35.71%
...d/InvenTree/order/migrations/0079_auto_20220304_0904.py	72	46	36.11%
.../InvenTree/common/migrations/0026_auto_20240608_1238.py	43	27	37.21%

Anexo F. Brechas y falsos negativos

Módulo	Pruebas Realizadas	Pruebas Exitosas	Porcentaje exito
Partes	8 de 8	7 de 8	87.5%
Stock	8 de 8	5 de 8	62.5%
Compra	8 de 8	7 de 8	87.5%
Venta	8 de 8	8 de 8	100%
Fabricación	8 de 8	8 de 8	100%
Empresas	5 de 5	5 de 5	100%
Roles	5 de 5	5 de 5	100%
Reportes	5 de 5	5 de 5	100%
TOTAL	55 PRUEBAS	50 PRUEBAS	91%

Anexo G. Tabla Resumen de Pruebas de Aceptación

Módulo	Archivos	Clases	Cas os	Base principal
Part	3	25	139	InvenTreeTestCase / InvenTreeAPITestCase
Stock	2	21	137	InvenTreeAPITestCase / InvenTreeTestCase
Order	3	20	185	InvenTreeAPITestCase / InvenTreeTestCase
Build	1	13	48	InvenTreeAPITestCase
TOTAL	9	79	509	—

Anexo H. Tabla Resumen Global de Pruebas Funcionales

```
desempeno_results.json
1 |
2 | "suite": "test_sistema_desempeno_inventree_suite.py",
3 | "tipo": "Desempeno",
4 | "timestamp": "2026-07-18T17:57:07.789210",
5 | "summary": {
6 |   "pass": 5,
7 |   "fail": 0,
8 |   "total": 5
9 | },
10 | "cases": [
11 |   {
12 |     "id": "PERF-01",
13 |     "desc": "GET /part/?limit=100 responde en < 2.0s",
14 |     "status": "PASS",
15 |     "detail": "HTTP 200 | tiempo=0.276s (umbral 2.0s)"
16 |   },
17 |   {
18 |     "id": "PERF-02",
19 |     "desc": "POST /stock/ responde en < 2.0s",
20 |     "status": "PASS",
21 |     "detail": "HTTP 201 | tiempo=0.329s (umbral 2.0s)"
22 |   },
23 |   {
24 |     "id": "PERF-03",
25 |     "desc": "10 GET /part/ concurrentes: todas exitosas y total < 10.0s",
26 |     "status": "PASS",
27 |     "detail": "exitosas=10/10 | tiempo_total=0.657s | tiempo_individual_promedio=0.643s"
28 |   },
29 |   {
30 |     "id": "PERF-04",
31 |     "desc": "p95 de 20 requests a GET /part/ < 1.5s",
32 |     "status": "PASS",
33 |     "detail": "p95=0.283s (umbral 1.5s) | min=0.262s max=0.363s | fallos=0/20"
34 |   },
35 |   {
36 |     "id": "PERF-05",
37 |     "desc": "Crear 20 StockItems en lote: promedio < 1.0s/item",
38 |     "status": "PASS",
39 |     "detail": "total=6.958s | promedio=0.348s/item (umbral 1.0s) | fallos=0/20"
40 |   }
41 | ]
42 |
```

Anexo I. JSON de Resultados de Pruebas de Sistema